DEPLOYMENT GUIDE

Deploying Matt Rushmer's Blog to AWS

A complete, step-by-step walkthrough for taking this local-first Next.js site from `npm run dev` to a live, AWS-hosted deployment using the project's own AWS CDK infrastructure.

ESTIMATED MONTHLY COST ONCE DEPLOYED

≈ $0.55 – $2.40 / month

Region: **ap-southeast-2** (Sydney)

Infrastructure: AWS CDK v2 (TypeScript), OpenNext for Next.js on Lambda

Covers: account setup, CLI/CDK install, deploy, Cognito admin user + MFA, verification, redeploys, teardown

Contents

1. 1. Before you start
2. 2. Create an AWS account and an IAM user
3. 3. Install and configure the AWS CLI
4. 4. Install Node.js, the project, and the CDK CLI
5. 5. Bootstrap your AWS account for CDK
6. 6. Set the deployment configuration variables
7. 7. Build the deployable artifacts
8. 8. Preview the changes (cdk synth / cdk diff)
9. 9. Deploy the stacks
10. 10. Confirm the cost-alert email subscription
11. 11. Create the admin user and set up MFA
12. 12. Finish wiring the server's CDN domain
13. 13. Visit the site and sign in
14. 14. Redeploying after a code change
15. 15. Monitoring and the budget alarm
16. 16. Tearing it all down
17. 17. Environment variable reference
18. 18. Troubleshooting
19. 19. Reference links

1 Before you start

This project is **local-first**: everything works on your own machine with zero AWS access via `npm install && npm run seed && npm run dev`. Deploying to AWS is a separate, optional step that this guide walks through end to end. The infrastructure is written as four AWS CDK stacks (`infra/lib/stacks/`) and already `cdk synth`-clean — nothing in this guide requires writing new code, only running commands against an AWS account you control.

> **THIS CREATES BILLABLE AWS RESOURCES**
>
> Every step from Deploy the stacks onward creates real AWS resources. At personal-site traffic the whole stack costs well under $3/month (see the cost box on the cover page), and a CloudWatch budget alarm is deployed automatically to flag anything unexpected — but you should still only run `cdk deploy` when you intend to.

What you'll end up with:

- A DynamoDB table for content, an S3 bucket for photos behind CloudFront, and a Lambda that serves the whole Next.js app.
- A Cognito user pool with a single admin account, MFA required.
- A CloudWatch budget alarm and a 5xx-error alarm, both emailing you.
- A live HTTPS URL (on the default `*.cloudfront.net` domain — no custom domain is configured by this project yet).

2 Create an AWS account and an IAM user

If you don't already have an AWS account, create one at aws.amazon.com/signup. You'll need a credit card and a phone number for verification.

> **DON'T USE YOUR ROOT ACCOUNT DAY TO DAY**
>
> AWS's own guidance is to create a separate IAM user (or IAM Identity Center user) for everyday work and keep the root account locked away for account-level tasks only. See IAM best practices.

Create an IAM user for deployment

1. Sign in to the IAM console as the root user (first time only).
2. Go to **Users** → **Create user**. Name it something like `blog-deployer`.
3. Attach the `AdministratorAccess` managed policy for now — CDK deploys create IAM roles, Lambda functions, S3 buckets, CloudFront distributions, Cognito pools, and more, and scoping a minimal policy for all of that is a project of its own. Once the stack is stable, you can tighten this.
4. Under **Security credentials**, create an **access key** (choose "Command Line Interface (CLI)" as the use case). Save the Access Key ID and Secret Access Key somewhere safe — the secret is shown only once.

Full walkthrough: Creating IAM users.

3 Install and configure the AWS CLI

Install the AWS CLI v2 for your platform: Installing the AWS CLI.

```
aws --version
# should print something like aws-cli/2.x.x
```

Configure it with the access key from Step 2:

```
aws configure
# AWS Access Key ID:     <paste>
# AWS Secret Access Key: <paste>
# Default region name:   ap-southeast-2
# Default output format: json
```

Verify it works:

```
aws sts get-caller-identity
```

Full reference: Configuring the AWS CLI.

4 Install Node.js, the project, and the CDK CLI

1. Install Node.js 22 or later from nodejs.org (the project pins Lambda runtimes to `nodejs22.x`).
2. Clone the repository and install dependencies:

   ```
   git clone https://github.com/MattRushmer/MyBlog.git
   cd MyBlog
   npm install
   ```

   `npm install` installs both the Next.js app's dependencies and the `infra/` CDK workspace's dependencies in one step (they're linked as npm workspaces).
3. Install the CDK CLI globally (or use `npx cdk` everywhere below instead):

   ```
   npm install -g aws-cdk
   cdk --version
   ```

Background on the CDK toolkit: Getting started with the AWS CDK.

5 Bootstrap your AWS account for CDK

CDK needs a small set of support resources (an S3 bucket for deployment assets, IAM roles) in your target account/region before it can deploy anything. This is a one-time step per account + region:

```
cdk bootstrap aws://<YOUR_ACCOUNT_ID>/ap-southeast-2
# find YOUR_ACCOUNT_ID with: aws sts get-caller-identity --query Account --output text
```

Details: Bootstrapping your AWS environment.

6 Set the deployment configuration variables

`infra/bin/app.ts` reads these as real shell environment variables (not from `.env.local` — that file configures the *application*, this configures the *CDK app* that builds the infrastructure). All of them have safe defaults for `cdk synth`, but you should set them explicitly for a real deploy:

Variable	Purpose	Example
`STAGE`	Suffixes every stack/resource name	`prod`
`AWS_REGION`	Deploy region	`ap-southeast-2`
`ALERT_EMAIL`	Receives budget + 5xx-alarm emails	`you@example.com`
`CONTENT_TABLE_NAME`	DynamoDB content table name (optional, has a default)	`blog-content-prod`
`ISR_TABLE_NAME`	DynamoDB ISR cache table name (optional)	`blog-isr-prod`
`PHOTOS_BUCKET_NAME`	S3 bucket name (optional — defaults to a name derived from your account ID, which is globally unique)	`blog-photos-prod-123456789012`

Set them for your current shell session (PowerShell example — adjust for bash/zsh):

```
$env:STAGE = "prod"
$env:AWS_REGION = "ap-southeast-2"
$env:ALERT_EMAIL = "you@example.com"
```

> **ONLY ALERT_EMAIL AND STAGE ARE STRICTLY REQUIRED**
>
> The table/bucket names fall back to sensible auto-generated defaults (`blog-content-<stage>` , etc., with the bucket name including your AWS account ID so it's globally unique). Set `ALERT_EMAIL` to a real address you can check — you'll need to confirm an SNS subscription email after the first deploy.

7 Build the deployable artifacts

Two things need building before CDK can package them: the OpenNext bundle (the Next.js app, compiled for Lambda) and the `sharp` image library's native binary for Lambda's `linux/arm64` runtime (the `infra` deploy scripts build this one automatically, so you only need the first step yourself):

```
npm run build:open-next
# produces .open-next/ — the server function, static assets, the
# revalidation function, etc. SiteStack reads directly from this folder.
```

> **RUN THIS AGAIN AFTER EVERY CODE CHANGE**
>
> `.open-next/` is a build output (git-ignored) — it reflects whatever source code was checked out when you last ran this command, not automatically your latest edits. Re-run it before every `cdk diff` or `cdk deploy` you expect to reflect new code.

8 Preview the changes

Sanity-check the whole stack synthesizes with no AWS access required:

```
npm run infra:synth
```

Once you have real credentials configured (Steps 2–3), preview exactly what a deploy would create or change:

```
npm run infra:diff
```

The first time, this will show every resource as new (DynamoDB tables, the S3 buckets, the Cognito pool, the CloudFront distribution, the Lambda functions, IAM roles, the SNS topic, the budget). Review it — this is the last point before anything billable gets created.

9 Deploy the stacks

```
npm --prefix infra run deploy -- --all
```

This runs the `sharp` layer build automatically, then `cdk deploy --all`, which deploys `DataStack`, `AuthStack`, `SiteStack`, and `OpsStack` in the correct dependency order (CDK resolves this automatically from the cross-stack references — `SiteStack` depends on tables from `DataStack` and the user pool from `AuthStack`, for instance). CDK will prompt you to confirm IAM changes for each stack — review and approve.

This takes several minutes, mostly CloudFront distribution creation. When it finishes, each stack prints its outputs — note down:

- **SiteUrlOutput** (from `BlogSiteStack-<stage>`) — your live URL.
- **ServerFunctionNameOutput** — needed in Step 12.
- **UserPoolIdOutput** and **UserPoolClientIdOutput** (from `BlogAuthStack-<stage>`) — needed in Step 11.

If you ever lose these, retrieve them again with:

```
aws cloudformation describe-stacks --stack-name BlogSiteStack-prod --query "Stacks[0].Outputs"
aws cloudformation describe-stacks --stack-name BlogAuthStack-prod --query "Stacks[0].Outputs"
```

10 Confirm the cost-alert email subscription

`OpsStack` creates an SNS topic subscribed to `ALERT_EMAIL` and attaches it to the CloudFront 5xx-error alarm. AWS sends a confirmation email immediately after deploy — **open it and click "Confirm subscription"**, or you won't actually receive alerts.

11 Create the admin user and set up MFA

Self-signup is disabled — the admin account has to be created directly via the AWS CLI, and because the pool requires TOTP MFA for every user, the first sign-in has to walk through MFA setup too. There's no UI for this (no Cognito Hosted UI is deployed — the app talks to Cognito directly), so it's all CLI, once, ever.

11.1 — Create the user

```
aws cognito-idp admin-create-user \
  --user-pool-id <USER_POOL_ID> \
  --username admin \
  --temporary-password "TempPass!23456" \
  --message-action SUPPRESS
```

Use a strong temporary password meeting the pool's policy (12+ characters, upper + lower + digit + symbol). `--message-action SUPPRESS` is required — the pool has no verified email/phone to send an invite to.

11.2 — Sign in once to set a permanent password

```
aws cognito-idp initiate-auth \
  --client-id <USER_POOL_CLIENT_ID> \
  --auth-flow USER_PASSWORD_AUTH \
  --auth-parameters USERNAME=admin,PASSWORD="TempPass!23456"
```

The response has `"ChallengeName": "NEW_PASSWORD_REQUIRED"` and a `"Session"` string. Copy that session value, then:

```
aws cognito-idp respond-to-auth-challenge \
  --client-id <USER_POOL_CLIENT_ID> \
  --challenge-name NEW_PASSWORD_REQUIRED \
  --session "<SESSION_FROM_PREVIOUS_RESPONSE>" \
  --challenge-responses USERNAME=admin,NEW_PASSWORD="YourRealPassword!456"
```

11.3 — Set up the authenticator app (TOTP)

Because MFA is required and this user has none configured yet, the response to 11.2 comes back with `"ChallengeName": "MFA_SETUP"` and a new `"Session"`. Use it to get a TOTP secret:

```
aws cognito-idp associate-software-token \
  --session "<SESSION_FROM_11.2>"
```

The response includes a `"SecretCode"` (a base32 string) and another new `"Session"`. Open an authenticator app (Google Authenticator, Authy, 1Password, etc.), choose "enter a setup key manually" / "can't scan the QR code", and enter the `SecretCode` — the app will start generating 6-digit codes.

Verify the code it just generated:

```
aws cognito-idp verify-software-token \
  --session "<SESSION_FROM_ASSOCIATE_STEP>" \
  --user-code "123456"
```

A successful response has `"Status": "SUCCESS"` and yet another new `"Session"`.

11.4 — Complete sign-in

```
aws cognito-idp respond-to-auth-challenge \
  --client-id <USER_POOL_CLIENT_ID> \
  --challenge-name MFA_SETUP \
  --session "<SESSION_FROM_VERIFY_STEP>" \
  --challenge-responses USERNAME=admin
```

This should finally return a real `AuthenticationResult` with tokens — confirming the account is fully set up. From now on, signing in (through the deployed site's own `/admin/login` page) only needs the username, the real password from 11.2, and a fresh 6-digit code from the authenticator app each time.

Reference: Adding MFA to a user pool, associate-software-token CLI reference.

12 Finish wiring the server's CDN domain

This is a known, deliberate gap documented directly in `infra/lib/stacks/site-stack.ts`: the server Lambda can't be told its own CloudFront domain at deploy time, because doing so would create a circular CloudFormation dependency (the Lambda is the distribution's origin, so the distribution can't be declared

as a dependency of the Lambda's own configuration). It has to be patched in once, manually, right after the first deploy.

1. Get the distribution domain from the `SiteUrlOutput` you noted in Step 9 (or re-fetch it per Step 9's last command).
2. Open the Lambda console, find the function named in `ServerFunctionNameOutput` (e.g. `blog-server-prod`), go to **Configuration** → **Environment variables** → **Edit**, and add two new variables without removing any existing ones:
   - `CDN_DOMAIN` = the distribution's domain (no `https://`, just the hostname, e.g. `d123abc456.cloudfront.net`)
   - `SITE_URL` = `https://` + that same domain
3. Save. The change takes effect on the next invocation — no redeploy needed.

> **THIS GETS REVERTED BY THE NEXT CDK DEPLOY**
> CDK manages this Lambda's environment variables declaratively from `SiteStack`'s source code, which doesn't know about `CDN_DOMAIN` / `SITE_URL` — so a future `cdk deploy` will overwrite the whole environment variable set back to only what's declared in code, silently dropping both. Redo this step after every redeploy (Step 14) until the code is changed to source these automatically (e.g. via a custom resource, or once a stable custom domain exists to hard-code instead).

13 Visit the site and sign in

1. Open the URL from `SiteUrlOutput` in a browser — you should see the home page, exactly like local mode but served from CloudFront.
2. Go to `/admin/login`, sign in with `admin` / the real password from Step 11.2.
3. Enter the 6-digit code from your authenticator app when prompted.
4. You're in. The content store is empty (a fresh DynamoDB table) — create your first story, project, certification, and link from here.

There is currently no script to migrate existing local (`./data`) content into AWS mode — if you've been writing content locally, you'll need to re-create it through this admin UI once, or write a one-off migration script against the `ContentStore` / `FileStorage` adapter interfaces (`src/adapters/`) yourself.

14 Redeploying after a code change

```
npm run build:open-next
npm --prefix infra run deploy -- --all
```

Re-run Step 12 afterward if the server function's environment variables got reset (they will be, every time, until that gap is closed in code — see the warning above).

15 Monitoring and the budget alarm

- **Budget:** `OpsStack` creates an AWS Budget capped at $10/month, emailing `ALERT_EMAIL` at 80% actual spend. View/adjust it in the Billing console → Budgets.
- **5xx alarm:** A CloudWatch alarm on the CloudFront distribution's 5xx error rate, emailing the same address via the SNS topic from Step 10.
- **Logs:** Lambda execution logs land in CloudWatch Logs under `/aws/lambda/blog-server-<stage>` and `/aws/lambda/blog-image-resize-<stage>` (actual log group name: look it up in the CloudWatch console).

16 Tearing it all down

```
npm --prefix infra run destroy -- --all
```

> **SOME RESOURCES SURVIVE DESTROY ON PURPOSE**
>
> The content DynamoDB table and the photos S3 bucket both have `RemovalPolicy.RETAIN` — deliberately, so a stack teardown can never silently delete real content. They'll remain in your account (and keep costing a small amount) after `cdk destroy` completes. To actually remove them: empty the S3 bucket first (a bucket with objects in it can't be deleted), then delete both the bucket and the DynamoDB table manually via the console or CLI.

Reference: AWS CDK CLI reference (see the `destroy` command).

17 Environment variable reference

Read by the CDK app (infra/bin/app.ts) — shell environment variables

Variable	Required	Default
`STAGE`	No	`dev`
`AWS_REGION`	No	`ap-southeast-2`
`ALERT_EMAIL`	Recommended	`placeholder@example.com` (won't receive real alerts)
`CONTENT_TABLE_NAME`	No	`blog-content-<stage>`
`ISR_TABLE_NAME`	No	`blog-isr-<stage>`
`PHOTOS_BUCKET_NAME`	No	`blog-photos-<stage>-<account-id>`

Read by the deployed app (set automatically by SiteStack, plus two manual ones)

Variable	Source
`DEPLOY_TARGET`	Set by CDK — always `aws`
`CONTENT_TABLE_NAME`, `ISR_TABLE_NAME`, `PHOTOS_BUCKET_NAME`	Set by CDK, from the values above
`COGNITO_USER_POOL_ID`, `COGNITO_CLIENT_ID`	Set by CDK automatically
`CDN_DOMAIN`, `SITE_URL`	**Manual** — Step 12, every deploy

18 Troubleshooting

Symptom	Likely cause / fix
Every page 500s after deploy	Usually `CDN_DOMAIN` / `SITE_URL` missing or a stale deploy of `.open-next` — redo Step 7 then Step 12.
`cdk deploy` fails with a bucket-name collision	S3 bucket names are globally unique across *all* AWS accounts. Set `PHOTOS_BUCKET_NAME` explicitly to something unambiguous.
`initiate-auth` returns `NotAuthorizedException`	Wrong password, or the account hasn't completed the Step 11 challenge sequence yet — Cognito won't allow normal sign-in until MFA setup is finished.
Admin login works but photo uploads fail	Check the browser console / Network tab for a 403 from the presigned S3 URL — usually means the S3 bucket's CORS policy doesn't match the origin you're testing from (CORS is currently `allowedOrigins: ["*"]`, so this should be rare).
`cdk bootstrap` / `cdk deploy` asks for approval repeatedly	Expected — every IAM-affecting change needs explicit approval unless you pass `--require-approval never` (not recommended for a stack with this many IAM statements).

19 Reference links

- **AWS account sign-up** — portal.aws.amazon.com/billing/signup
- **IAM best practices** — docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html
- **Creating IAM users** — docs.aws.amazon.com/IAM/latest/UserGuide/id_users_create.html
- **Installing the AWS CLI** — docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html
- **Configuring the AWS CLI** — docs.aws.amazon.com/cli/latest/userguide/cli-configure-quickstart.html
- **Node.js downloads** — nodejs.org
- **Getting started with the AWS CDK** — docs.aws.amazon.com/cdk/v2/guide/getting-started.html
- **CDK bootstrapping** — docs.aws.amazon.com/cdk/v2/guide/bootstrapping.html

- **CDK CLI reference** — docs.aws.amazon.com/cdk/v2/guide/cli.html
- **Adding MFA to a Cognito user pool** — docs.aws.amazon.com/cognito/latest/developerguide/user-pool-settings-mfa.html
- **Cognito admin-create-user CLI** — docs.aws.amazon.com/cli/latest/reference/cognito-idp/admin-create-user.html
- **Cognito associate-software-token CLI** — docs.aws.amazon.com/cli/latest/reference/cognito-idp/associate-software-token.html
- **CloudFormation describe-stacks CLI** — docs.aws.amazon.com/cli/latest/reference/cloudformation/describe-stacks.html
- **Lambda console** — console.aws.amazon.com/lambda
- **Billing → Budgets console** — console.aws.amazon.com/billing/home#/budgets
- **CloudWatch Logs console** — console.aws.amazon.com/cloudwatch/home#logsV2:log-groups
- **Project repository** — github.com/MattRushmer/MyBlog

Generated for the MyBlog project (github.com/MattRushmer/MyBlog). Reflects the AWS CDK infrastructure as of the commit this guide was built against — re-check `infra/lib/stacks/` and `docs/README.md` if the code has moved on since.